T1 - Tarea 1

Programación de Procesos en C

Memoria explicativa: qué se ha hecho, cómo funciona y qué está pasando de verdad

Criterios a), e), f), g), h)

Fecha de entrega: 13/10/2026 - 23:59

Archivos: **main.c, entrada.c/.h, proceso.c/.h, config.h, Makefile**

1. ¿Qué pide la tarea?

Hay que crear **un único programa en C** que cumpla estas Funcionalidades Requeridas (FR):

FR	Qué hay que hacer	Puntos
FR1	Crear un proceso (habrá dos: un **padre** y un **hijo**)	1
FR2	Pedir al usuario una variable	1
FR3	El proceso padre resta 5 a la variable	1
FR4	El proceso hijo le suma 4	1
FR5	Mostrar todos los valores por pantalla	1
-	Control de errores	2
-	Documentar el código	2

Y al final el enunciado lanza una pregunta trampa: «*¿Entiendes lo que está pasando? ¿De verdad?*». Esa pregunta es lo más importante de la tarea y se responde con detalle en el apartado 6.

2. Conceptos básicos (explicados fácil)

2.1. Programa vs. proceso

Un **programa** es un fichero con instrucciones guardado en el disco (como una receta de cocina escrita en un papel). Un **proceso** es ese programa **ejecutándose**: tiene su propia memoria, sus variables, su posición dentro del código... (es como alguien cocinando esa receta). Se puede tener la misma receta cocinándose en dos cocinas a la vez: dos procesos del mismo programa.

2.2. El PID

Cada proceso tiene un número de identificación único llamado **PID** (*Process ID*), como un DNI. En C se consulta con `getpid()`. Además, cada proceso conoce el PID de quien lo creó (su padre) con `getppid()` (PPID = *Parent PID*).

2.3. fork(): la fotocopiadora de procesos

`fork()` es la llamada al sistema que crea un proceso nuevo. Lo que hace es **clonar** el proceso que la llama: el nuevo proceso (el **hijo**) es una copia casi exacta del original (el **padre**): mismo código, mismas variables con los mismos valores y continúa ejecutando justo desde la línea siguiente al `fork()`.

Lo curioso es que `fork()` se llama **una vez** pero **devuelve dos veces** (una en cada proceso), y devuelve un valor distinto en cada uno. Así es como cada proceso sabe quién es:

Valor que devuelve fork()	Significado
Menor que 0 (-1)	Error: no se ha podido crear el hijo (no hay hijo)
Igual a 0	Estoy en el proceso **HIJO**
Mayor que 0	Estoy en el proceso **PADRE**, y el número es el PID del hijo

2.4. wait / waitpid y los procesos zombie

Cuando un hijo termina, el sistema guarda un pequeño registro con su código de salida hasta que el padre lo "recoge". Si el padre nunca lo recoge, el hijo se queda como **proceso zombie** (muerto, pero ocupando una entrada en la tabla de procesos). Con `waitpid()` el padre **espera** a que el hijo acabe y recoge su estado, evitando zombies.

3. El programa paso a paso

El programa está dividido en **módulos**. En C no existen las clases (eso es de C++ o Java); la forma profesional de organizar un programa en C es separarlo en varios archivos: cada módulo tiene un **.h** (la *interfaz*: qué funciones ofrece, como el "menú" de un restaurante) y un **.c** (la *implementación*: cómo se hacen, la "cocina"). Todo se compila en **un único ejecutable**, así que sigue siendo un único programa, como pide el enunciado.

Archivo	Qué contiene	Requisitos
config.h	Las constantes RESTA_PADRE (5) y SUMA_HIJO (4)	FR3, FR4
entrada.h / entrada.c	Pedir el número y validarlo: `leer_entero()`, `pedir_entero_hasta_valido()`	FR2, errores
proceso.h / proceso.c	`crear_proceso_hijo()`, `ejecutar_hijo()`, `ejecutar_padre()`, `esperar_hijo()`, `mostrar_resumen()`	FR1, 3, 4, 5
main.c	Coordina todo: llama a los módulos en orden	Todos
Makefile	Instrucciones para compilar todo con un solo comando: `make`	-

Cada **.h** lleva una *guarda de inclusión* (`#ifndef ENTRADA_H ... #endif`) para que no se incluya dos veces, y las funciones que solo se usan dentro de un módulo se marcan como `static` (privadas, como un método privado de una clase). Todos los archivos están comentados: cabecera, cada función y cada bloque.

Así queda **main.c**, que se lee casi como el enunciado:

```
if (pedir_entero_hasta_valido(&variable) != 0)   /* FR2 */
    return EXIT_FAILURE;
valor_original = variable;

pid = crear_proceso_hijo();                       /* FR1 */
if (pid < 0)  return EXIT_FAILURE;
if (pid == 0) ejecutar_hijo(&variable);           /* HIJO: FR4 (termina con exit) */

ejecutar_padre(&variable, pid);                   /* PADRE: FR3 */
if (esperar_hijo(pid) != 0) return EXIT_FAILURE;
mostrar_resumen(valor_original, variable);        /* FR5 */
```

3.1. config.h: las constantes

```
#define RESTA_PADRE 5   /* FR3: el proceso padre resta 5 */
#define SUMA_HIJO   4   /* FR4: el proceso hijo suma 4   */
```

Se usan constantes con nombre para el 5 y el 4. Así el código se entiende mejor y, si cambiase el enunciado, solo habría que tocar este archivo.

3.2. FR2: pedir la variable con control de errores (entrada.c)

Se ha hecho una función `leer_entero()`. En vez del típico `scanf("%d")` se usa `fgets()` (lee la línea entera como texto) + `strtol()` (convierte texto a número y avisa de los problemas). `scanf` tiene dos defectos: si escribes letras deja "basura" en la entrada y puede provocar un bucle infinito, y si el número es enorme no avisa.

```
errno = 0;
valor = strtol(buffer, &fin, 10);

if (fin == buffer)                      -> "Eso no es un número entero."
if (*fin != '\n' && *fin != '\0')       -> "Hay caracteres no válidos tras el número."
if (errno == ERANGE || fuera de int)    -> "Número fuera de rango."
if (valor-5 o valor+4 se desbordaría)   -> "Las operaciones se desbordarían."
```

La función `pedir_entero_hasta_valido()` la llama dentro de un bucle `do ... while`: si el usuario se equivoca, se le vuelve a preguntar hasta que escriba un número correcto. Si la entrada se cierra (Ctrl+D / EOF), el programa termina con error.

3.3. Antes del fork: fflush(stdout)

```
printf("[PADRE PID=%d] Valor original de la variable: %d\n", getpid(), variable);
fflush(stdout);   /* vaciar el búfer ANTES de clonar */
pid = fork();
```

`printf` no escribe al instante: guarda el texto en un **búfer** (una zona de memoria) y lo suelta más tarde. Como `fork()` copia toda la memoria, si el búfer tuviera texto pendiente ¡el hijo también lo tendría y se imprimiría dos veces! `fflush(stdout)` lo vacía antes de clonar y evita ese problema.

3.4. FR1: crear el proceso y separar padre e hijo (proceso.c)

```
pid_t crear_proceso_hijo(void) {
    fflush(stdout);
    pid_t pid = fork();
    if (pid < 0) perror("[ERROR] fork");   /* ERROR: no se pudo crear el hijo */
    return pid;
}

void ejecutar_hijo(int *variable) {        /* ===== HIJO ===== */
    *variable += SUMA_HIJO;                /* FR4: +4 */
    printf("[HIJO ...] Tras sumar 4 -> variable = %d\n", *variable);
    exit(EXIT_SUCCESS);                    /* el hijo termina aquí */
}

void ejecutar_padre(int *variable, pid_t pid_hijo) {   /* ===== PADRE ===== */
    *variable -= RESTA_PADRE;              /* FR3: -5 */
    printf("[PADRE ...] Tras restar 5 -> variable = %d\n", *variable);
}
```

Los dos procesos ejecutan **el mismo código** de `main()`, pero el `if (pid == 0)` hace que el hijo vaya a `ejecutar_hijo()` y el padre siga hacia `ejecutar_padre()`. El `exit()` del hijo es importante: sin él, el hijo volvería a `main()` y ejecutaría también el código del padre.

Las dos funciones reciben un **puntero** (`int *variable`) a la variable de `main()`; así modifican esa misma variable y se puede ver que padre e hijo tienen la **misma dirección** pero **valores distintos** (apartado 6.2).

3.5. El padre espera al hijo: esperar_hijo() (proceso.c)

```
do {
    terminado = waitpid(pid, &estado_hijo, 0);
} while (terminado == -1 && errno == EINTR);   /* reintentar si una señal lo interrumpe */

if (terminado == -1)            -> perror("waitpid"), error
if (WIFEXITED(estado_hijo))     -> muestra el código de salida del hijo
if (WIFSIGNALED(estado_hijo))   -> el hijo murió por una señal: error
```

3.6. FR5: resumen final

Al final el padre imprime un resumen con los tres valores (original, padre e hijo). Ojo: el padre **no puede leer** la variable del hijo (luego se explica por qué), así que el valor del hijo del resumen lo **calcula** el padre; el valor real del hijo es el que el propio hijo imprimió en su línea.

3.7. Resumen del control de errores

Error posible	Cómo se controla
El usuario escribe letras ("hola")	strtol no lee nada → mensaje y se vuelve a pedir
Número con basura detrás ("12abc")	Se comprueba lo que queda tras el número
Línea vacía	Se trata como "no es un número"
Número gigante (99999999999)	errno == ERANGE / fuera de INT_MIN..INT_MAX
Número que se desbordaría al operar (2147483647 + 4)	Se comprueba antes de operar
Línea demasiado larga	Se descarta el resto de la línea
Fin de entrada (Ctrl+D)	fgets devuelve NULL → fin con EXIT_FAILURE
fork() falla (sin memoria, límite de procesos)	pid < 0 → perror() y EXIT_FAILURE
waitpid() falla o es interrumpido	Se reintenta si EINTR; si no, perror()
El hijo termina mal o por una señal	WIFEXITED / WEXITSTATUS / WIFSIGNALED

4. Cómo lo he compilado y ejecutado

`fork()` es una llamada de sistemas **Linux/Unix** (POSIX) y **no existe en Windows**. El ordenador usado es Windows 11 y no tenía compilador de C, pero sí tenía **Docker** con la imagen oficial `gcc`, que es un Linux pequeño con el compilador ya instalado. Así que se ha compilado y ejecutado dentro de ese contenedor:

```
docker run --rm -it -v "%cd%:/src" -w /src gcc:latest bash

# ya dentro del Linux del contenedor:
make          # compila todos los módulos con el Makefile
./procesos    # ejecuta el programa

# o, sin make:
gcc -Wall -Wextra -o procesos main.c entrada.c proceso.c
```

Las opciones `-Wall -Wextra` activan todos los avisos del compilador. El programa compila **sin ningún aviso**. En cualquier Linux (o WSL, o una máquina virtual Ubuntu) basta con las dos últimas líneas.

5. Resultados de las pruebas (salida real)

Prueba 1: funcionamiento normal con el número 10

```
=== T1 - Programación de Procesos en C ===

Introduce un número entero: 10

[PADRE  PID=23] Valor original de la variable: 10
[PADRE  PID=23] Dirección de la variable     : 0x7fffc8fc5ff4
[PADRE  PID=23] He creado un hijo con PID=24
[PADRE  PID=23] Tras restar 5 -> variable = 5   (dir. 0x7fffc8fc5ff4)
[HIJO   PID=24, PPID=23] He recibido la variable con valor: 10
[HIJO   PID=24] Tras sumar 4 -> variable = 14   (dir. 0x7fffc8fc5ff4)
[PADRE  PID=23] El hijo 24 terminó con código 0

=== RESUMEN ===
Valor original           : 10
Valor en el padre (-5)   : 5
Valor en el hijo  (+4)   : 14

El padre sigue teniendo 5: el +4 del hijo NO le afectó,
porque cada proceso tiene su propia copia de la memoria.
```

¡Fíjate! El padre ya había restado 5 (variable = 5) y, aun así, el hijo dice que ha recibido un **10** y lo convierte en **14** (no en 9). Y los dos imprimen **la misma dirección de memoria** (0x7fffc8fc5ff4) pero con **valores distintos**. Esto es justo lo que explica el apartado 6.

Prueba 2: control de errores

Se escribieron, uno tras otro: `hola`, `12abc`, una línea vacía, `99999999999`, `2147483647` y finalmente `-3`:

```
  [ERROR] Eso no es un número entero.                          <- hola
  [ERROR] Hay caracteres no válidos tras el número.            <- 12abc
  [ERROR] Eso no es un número entero.                          <- (vacía)
  [ERROR] Número fuera de rango (-2147483648 .. 2147483647).   <- 99999999999
  [ERROR] Con ese número las operaciones se desbordarían.      <- 2147483647
...
```

```
[PADRE  PID=16] Tras restar 5 -> variable = -8
[HIJO   PID=17] Tras sumar 4 -> variable = 1
```

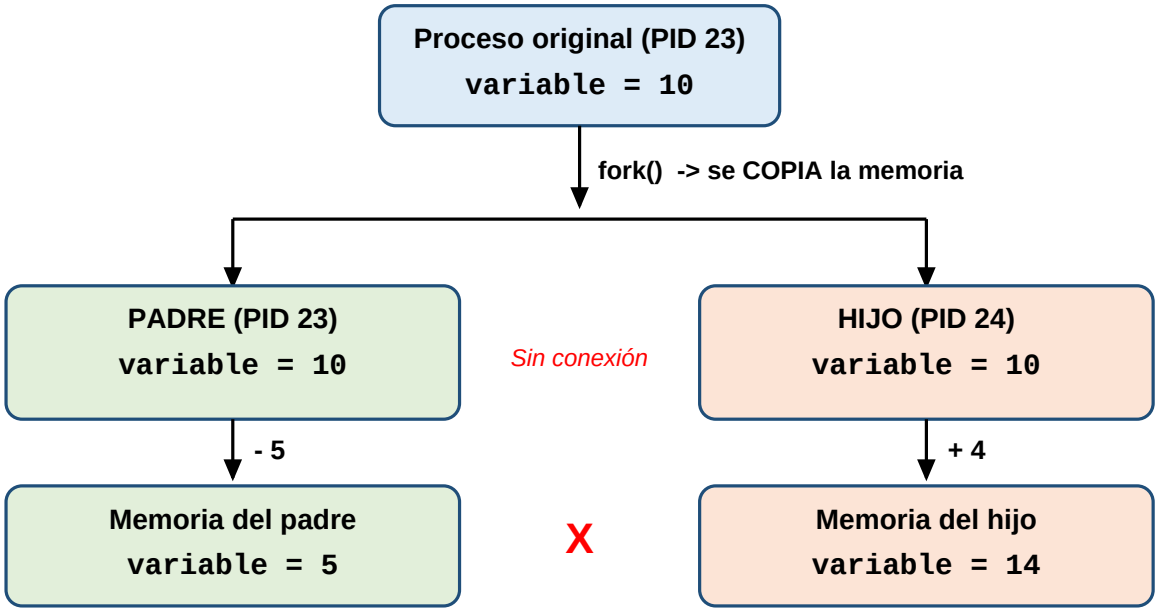
El programa no se rompe con ninguna entrada incorrecta: avisa y vuelve a preguntar. Con `-3` funciona bien también con negativos (-3-5 = -8 y -3+4 = 1).

Prueba 3: entrada cerrada (EOF)

```
[ERROR] No se pudo leer la entrada (EOF). Fin del programa.
(código de salida del programa: 1)
```

6. «¿Entiendes lo que está pasando? ¿De verdad?»

Mucha gente espera que el resultado sea algo como *10 → 5 → 9* (primero el padre resta 5 y luego el hijo suma 4 al resultado), como si compartieran la variable. **No es así.** Lo que pasa realmente es esto:



6.1. Cada proceso tiene SU PROPIA copia de la variable

En el momento del `fork()` el sistema operativo hace una "fotocopia" de la memoria del padre para el hijo. A partir de ese instante hay **dos variables distintas** que se llaman igual, una en cada proceso. Cuando el padre resta 5, cambia la suya; cuando el hijo suma 4, cambia la suya. Ninguno ve los cambios del otro. Por eso el resultado es **padre = 5** e **hijo = 14**, y nunca 9.

Es como si fotocopias un examen y se lo das a un compañero: si tú escribes en tu hoja, la fotocopia de tu compañero no cambia.

6.2. ¿Y por qué la dirección de memoria es la misma?

Esto es lo que más despista: el padre y el hijo imprimen `0x7fffc8fc5ff4` y aun así tienen valores distintos. La razón es la **memoria virtual**: las direcciones que ve un programa no son posiciones reales de la RAM, sino direcciones "de mentira" dentro de su propio espacio. Cada proceso tiene su propio mapa que traduce esas direcciones a la RAM real. Como el hijo es un clon, tiene el **mismo mapa de direcciones**, pero el sistema operativo las traduce a **zonas físicas distintas**.

Es como dos personas que viven en "calle Mayor, 5"... pero una en Madrid y otra en Sevilla: la misma dirección, casas diferentes.

6.3. Copy-on-write: la copia es "perezosa"

Copiar toda la memoria en cada `fork()` sería lento, así que Linux hace un truco llamado **copy-on-write** (copiar al escribir): al principio padre e hijo **comparten** las mismas páginas de RAM marcadas como "solo lectura". En el momento en que uno de los dos intenta **modificar** algo (por ejemplo, `variable -= 5`), el sistema duplica solo esa página y le da a ese proceso su copia

privada. Para el programador el efecto es el mismo: las variables son independientes.

6.4. ¿Quién va primero, el padre o el hijo?

Después del `fork()` hay dos procesos **ejecutándose a la vez** y es el **planificador** del sistema operativo quien decide cuál usa la CPU en cada momento. **No hay ningún orden garantizado.** En las pruebas (30 ejecuciones seguidas) el padre imprimió primero todas las veces, pero en otro ordenador, con otra carga de trabajo o con varios núcleos, las líneas del hijo podrían salir antes o mezcladas con las del padre. Por eso el resultado de cada uno nunca depende del otro, y por eso el padre usa `waitpid()` antes de mostrar el resumen: es la única forma de asegurarse de que el hijo ya ha acabado.

6.5. fork() devuelve dos veces

Se llama a `fork()` en una sola línea, pero a partir de ahí esa línea "vuelve" en dos procesos. En el padre, la variable `pid` vale 24 (el PID del hijo); en el hijo vale 0. Por eso el mismo `if (pid == 0)` es verdadero en uno y falso en otro. Además, el hijo muestra `PPID=23`: su padre es el proceso 23.

6.6. Entonces, ¿cómo se comunicarían si quisieran?

Como no comparten variables, si el padre quisiera saber el valor del hijo necesitaría un mecanismo de **comunicación entre procesos** (IPC): tuberías (`pipe()`), memoria compartida (`shmget()`/`mmap()`), ficheros, señales... o, para algo muy pequeño, el código de salida del hijo que recoge `waitpid()`. Si se quisiera de verdad compartir variables, se usarían **hilos** (*threads*) en lugar de procesos.

7. Comprobación de los requisitos

Requisito	Dónde está en el código	Estado
FR1: crear un proceso	`crear_proceso_hijo()` en proceso.c y `if (pid == 0)` en main.c	Hecho
FR2: pedir una variable	Módulo entrada.c: `leer_entero()` y `pedir_entero_hasta_valido()`	Hecho
FR3: el padre resta 5	`variable -= RESTA_PADRE;` en `ejecutar_padre()` (proceso.c)	Hecho
FR4: el hijo suma 4	`variable += SUMA_HIJO;` en `ejecutar_hijo()` (proceso.c)	Hecho
FR5: mostrar todos los valores	Valor original, valor de cada proceso (con PID y dirección) y resumen final	Hecho
Control de errores	Validación completa de la entrada, desbordamiento, fork, waitpid y estado del hijo (ver 3.7)	Hecho
Documentación del código	Cabecera en cada archivo, comentario de cada función en los .h y de cada bloque en los .c; esta memoria	Hecho

8. Conclusión

Se ha creado un programa en C que pide un número, crea un proceso hijo con `fork()`, hace que el padre reste 5 y el hijo sume 4, muestra todos los valores y controla todos los errores razonables. Se ha compilado sin avisos y se ha probado con entradas correctas e incorrectas.

La lección de verdad es que **un proceso hijo no comparte memoria con su padre**: recibe una copia en el momento del `fork()` y, desde ahí, cada uno va por su cuenta. Por eso, partiendo de 10, el padre termina con **5** y el hijo con **14**, aunque las variables se llamen igual e incluso tengan la misma dirección (virtual) de memoria. Y además el orden en que se ejecutan no está garantizado: lo decide el sistema operativo. **Sí, ahora sí se entiende lo que está pasando.**